

Edge Audio Intelligence Framework

Linux Setup and Deployment Guide

1. Overview

This document provides step-by-step instructions to set up and run the Edge Audio Intelligence Framework on a fresh Linux system. The framework runs 14 parallel AI tasks for real-time audio analysis — entirely offline with zero cloud dependency.

Supported distributions: Ubuntu 20.04+, Debian 11+, Raspberry Pi OS (64-bit).

2. Prerequisites

Before starting, ensure you have:

- A Linux machine (physical or VM) with at least 4 GB RAM
- Sudo/root access for installing system packages
- Internet access for the initial model download (approximately 1.1 GB)
- Python 3.9 or later installed
- A working microphone (for live analysis) or WAV audio files (for file analysis)

3. Step 1: Install System Dependencies

Linux requires native C libraries for audio capture and WAV file processing. Run the following command to install all required system packages:

```
sudo apt-get update
sudo apt-get install -y python3 python3-pip python3-venv python3-dev git
\
libportaudio2 libsndfile1 python3-pyaudio portaudio19-dev ffmpeg
```

Note: If you are using a Raspberry Pi, these same commands apply. The framework is compatible with ARM64 architecture.

4. Step 2: Get the Project

Option A: Clone from GitHub

```
git clone <your-github-repo-url> edge_audio_framework
cd edge_audio_framework
```

Option B: Copy from USB / Network Drive

If deploying to an air-gapped machine, copy the entire project folder (including the models/ directory) from a previously set up machine:

```
cp -r /media/usb/edge_audio_framework ~/edge_audio_framework
cd ~/edge_audio_framework
```

Note: If you copy the models/ folder from a previous setup, you can skip Step 5 (Download AI Models) entirely.

5. Step 3: Create Virtual Environment

Create an isolated Python environment to avoid conflicts with system packages:

```
python3 -m venv venv
source venv/bin/activate
```

Note: You must run "source venv/bin/activate" every time you open a new terminal session.

6. Step 4: Install Python Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

This installs PyTorch, HuggingFace Transformers, SpeechBrain, Librosa, and all other required Python libraries. This may take 5-10 minutes depending on your internet speed.

7. Step 5: Download AI Models

This one-time command downloads all 9 AI model checkpoints (approximately 1.1 GB) into the local models/ directory inside the project:

```
python download_models.py
```

The downloader will display progress for each model:

- [1/9] Whisper base (ASR) -- 74 MB
- [2/9] SpeechBrain VoxLingua107 (Language ID) -- 80 MB
- [3/9] Accent ID -- derived from Language ID (no download)
- [4/9] SUPERB HuBERT-base (Emotion) -- 90 MB

- [5/9] MIT AST AudioSet (Environmental Sound) -- 350 MB
- [6/9] MIT AST Speech Commands (Keyword Spotting) -- 350 MB
- [7/9] DistilHuBERT GTZAN (Music Genre) -- 90 MB
- [8/9] SpeechBrain ECAPA VoxCeleb (Speaker ID) -- 80 MB
- [9/9] Silero VAD (Voice Activity) -- 5 MB

Note: After this step, the framework operates 100% offline. No internet is required at runtime.

8. Step 6: Verify Setup

Run a quick test to confirm everything is working:

```
python fast_run.py --agent --tasks vad,asr --duration 5
```

If successful, you will see a 3-2-1 countdown, record 5 seconds of audio, and display results for Voice Activity Detection and Speech-to-Text.

9. Running Live Microphone Analysis

The framework processes live microphone audio in 4 recommended batches. Running in batches keeps memory usage low on edge hardware.

Batch 1: Speech and Keyword Detection

```
python fast_run.py --agent --tasks vad,asr,keyword_spotting
```

Batch 2: Identity, Emotion and Quality

```
python fast_run.py --agent --tasks speaker_id,emotion,speech_quality
```

Batch 3: Language and Environmental Audio

```
python fast_run.py --agent --tasks lang_accent_id,esc,music_genre
```

Batch 4: Signal Processing and Anomalies

```
python fast_run.py --agent --tasks  
audio_quality_monitoring,anomaly_detection,impulse_event
```

Run All Tasks at Once

```
python fast_run.py --agent --tasks all
```

Note: Running all tasks at once requires approximately 4 GB of RAM.

10. Running Audio File Analysis

To analyze a pre-recorded WAV file instead of using a live microphone:

```
python fast_run_file.py --file "sample_data/test_audio.wav" --agent --tasks vad,asr,emotion
python fast_run_file.py --file "/home/user/recordings/call.wav" --agent -tasks all
```

Supported audio format: WAV (mono or stereo, any sample rate). The framework automatically resamples to 16kHz.

11. Additional Command Options

Option	Command
Record 10 seconds	python fast_run.py --duration 10 --agent
Use a specific microphone	python fast_run.py --device 2 --agent
List available microphones	python fast_run.py --list-devices
Skip model preloading	python fast_run.py --no-prewarm --agent
Force offline mode	export TRANSFORMERS_OFFLINE=1
View event history	cat agent_memory/events.jsonl

12. Task Name Reference

Use these exact names with the --tasks flag (comma-separated, no spaces):

Task Name	Description
vad	Voice Activity Detection
asr	Automatic Speech Recognition (Whisper)
keyword_spotting	Keyword / Command Detection
speaker_id	Speaker Identification and Counting
emotion	Emotion Recognition
speech_quality	Speech Quality (DNSMOS + SNR)
lang_accent_id	Language and Accent Identification
esc	Environmental Sound Classification
acoustic_event_detection	Acoustic Event Detection
audio_quality_monitoring	Audio Quality Monitor (MOS, SNR, Clipping)
music_speech_detection	Music vs. Speech Classifier
anomaly_detection	Audio Anomaly Detector
impulse_event	Impulse / Gunshot Event Detector
music_genre	Music Genre Classification

13. Offline / Air-Gapped Deployment

To deploy on a machine with no internet access:

1. Set up the framework on an internet-connected machine following steps 1-5 above.
2. Copy the entire edge_audio_framework/ folder (including models/) to a USB drive.
3. On the target machine, copy the folder from USB and create a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Set the offline environment variable and run:

```
export TRANSFORMERS_OFFLINE=1
python fast_run.py --agent --tasks vad,asr,emotion
```

14. Troubleshooting

Microphone not detected

```
python fast_run.py --list-devices
python fast_run.py --device <ID> --agent --tasks vad,asr
```

PyAudio installation fails

```
sudo apt-get install -y portaudio19-dev python3-dev
pip install pyaudio
```

Out of memory on low-RAM devices

Run tasks in smaller batches (2-3 tasks at a time):

```
python fast_run.py --agent --tasks vad,asr
```

Permission denied on audio device

Add your user to the audio group:

```
sudo usermod -aG audio $USER
```

Then log out and log back in.